

FeedBackIQ A Review Analysis and Sales Impact Prediction System for E-Commerce Platforms

**DISSERTATION SUBMITTED
IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS
OF THE DEGREE OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

BY

DEVDEEP SINGH - AJU/221170

AMAR DAS - AJU/220948

RAJ KUMAR - AJU/221714

SUDHANSHU SINGH - AJU/221513

RISHI RIKHI- AJU/221177



**ARKA JAIN
University**
Jharkhand



**Under the supervision of
Dr. Megha Sinha
Department of Computer Science and IT**

**School of Engineering & I.T,
ARKA JAIN University, Jharkhand
(2022-2026)**



ARKA JAIN
University
Jharkhand



CERTIFICATE

23rd April 2026

This is to certify that project entitled **“FeedBackIQ A Review Analysis and Sales Impact Prediction System for E-Commerce Platforms”** has been submitted to the Department of Computer Science and Engineering. ARKAJAIN University, Jharkhand for the fulfillment of the requirement for the award of degree of BTech in Computer Science & Engineering” by following student of final year BTech (Computer Science & Engineering).

DEVDEEP SINGH - AJU/221170

AMAR DAS - AJU/220948

RAJ KUMAR - AJU/221714

SUDHANSHU SINGH - AJU/221513

RISHI RIKHI- AJU/221177

Dr. Megha Sinha
(Project Guide)
School of Engineering & IT

Dr. Ashwini Kumar
Assistant Dean
School of Engineering & IT

Asst. Dean
School of Engineering & IT
ARKA JAIN University



ARKA JAIN
University
Jharkhand



DECLARATION BY THE STUDENT

I hereby declare that the project report entitled “**FeedbackIQ A Review Analysis and Sales Impact Prediction System for E-Commerce Platforms**” submitted by us to **ARKA JAIN UNIVERSITY**; Jharkhand in partial fulfillment of the requirement by the award of the degree of **B.Tech. in Computer Science and Engineering** is a record of bonafide project work carried out by us under the guidance of dre. **Megha Sinha**. I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree in this university or any other institute or university.

We will be solely responsible if any kind of plagiarism is found.

Date:- 23 / 0 4 / 2 0 2 6

ARKA JAIN University

Devdeep Singh

Devdeep Singh
AJU/221170

Raj Kumar

Raj Kumar
AJU/221714

Rishi Rikhi

Rishi Rikhi
AJU/221177

Amar Das

Amar das
AJU/220948

Sudhanshu Singh

Sudhanshu Singh
AJU/221513

ACKNOWLEDGMENT

We like to share our sincere gratitude to all those who help us in completion of this project. During the work we faced many challenges due to our lack of knowledge and experience but these people help us to get over from all the difficulties and in final completion of our idea to a shaped sculpture.

We would like to thank **Dr. Ashwini Kumar** sir for his governance and guidance, because of which our whole team was able to learn the minute aspects of a project work.

We would also like to thank our Project Supervisor **Dr. Megha Sinha** for her continuous help and monitoring during the project work.

In the last we would like to thank the management of ARKA JAIN UNIVERSITY for providing us such an opportunity to learn from their experiences.

All of our team is thankful to all the faculties and staff of Department of Computer Science and Engineering, AJU, Jharkhand, for their help and support towards this project and our team.

We are also thankful to our whole class and most of all to our parents who have inspired to face all the challenges and will all the hurdles in life.

April 23, 2026
ARKA JAIN University

Devdeep Singh (AJU/221170)

Amar Das (AJU/220948)

Raj Kumar (AJU/221714)

Sudhanshu Singh (AJU/221513)

Rishi Rikhi (AJU/221177)

ABSTRACT

This project presents a web-based system for automated extraction and analysis of product reviews from multiple e-commerce platforms such as Amazon, Flipkart, Myntra, and Snapdeal. In today's digital marketplace, customer reviews play a crucial role in influencing purchasing decisions and determining product success. However, manually analyzing a large volume of reviews is time-consuming and inefficient.

The proposed system uses web scraping techniques to collect real-time product reviews and applies Natural Language Processing (NLP) using the VADER sentiment analysis model to classify reviews into positive, negative, and neutral categories. In addition to sentiment classification, the system identifies frequently used keywords to highlight common customer opinions and issues.

The application also generates visual insights through graphs such as sentiment distribution and keyword frequency, making the analysis more intuitive and user-friendly. A key feature of this system is the prediction of sales impact based on customer sentiment, which helps in understanding how negative or neutral feedback affects product performance over time.

The system is developed using Flask for backend processing, Selenium for web scraping, and Matplotlib for data visualization. This project provides a scalable and efficient solution for both customers and businesses to gain meaningful insights from product reviews.

TABLE OF CONTENTS

1. DECLARATION BY THE STUDENT	3
2. CERTIFICATE	2
3. ACKNOWLEDGMENT	4
4. ABSTRACT	5
5. INTRODUCTION	8
5.1 About	9
5.2 Objectives	9
5.3 Key Features of the Review Analysis and Sales Impact Prediction System	10
5.3.1 User Input and Product Handling	10
5.3.2 Review Extraction System	10
5.3.3 Sentiment Analysis System	10
5.3.4 Keyword Analysis System	10
5.3.5 Data Visualization	11
5.3.6 Rating and Sales Impact Analysis	11
5.3.7 System Interface and Usability	11
6. LITERATURE REVIEW	12
7. TECHNOLOGY STACK	13
7.1 Frontend Development	13
7.2 Backend Development	13
7.3 Web Scraping	13
7.4 Data Processing and Analysis	13
7.5 Data Visualization	13
7.6 Sales Prediction Module	13
7.7 Automation and Background Processing	14
7.8 Data Handling and Storage	14
8. PROJECT DEVELOPMENT PHASES	15
8.1 Requirement Gathering and Analysis	15
8.2 System Design	15
8.3 Development and Integration	15
8.4 Visualization and Feature Enhancement	16
8.5 Testing and Debugging	16
9. SYSTEM DESIGN AND CODE IMPLEMENTATION	17
9.1 Landing Page	17
9.2 Result Page	18
9.3 Sales Impact Module	18
9.4 Review Extraction Module	18
9.5 Code Implementation	19-53
○ Scraper.py	
○ Analyzer.py	
○ Sales.py	
○ app.py	
○ index.html	
10. CONCLUSION	55
11. REFERENCES	39

LIST OF FIGURES

Figure 1: FeedbackIQ Landing Page Interface	27
Figure 2: Product Search and Input Module	28
Figure 3: Product Review Analysis Result Page	29
Figure 4: Sentiment Analysis Pie Chart	30
Figure 5: Keyword Frequency Bar Graph	31
Figure 6: Sentiment-Based Keyword Analysis Section	32
Figure 7: Sales Impact Prediction Dashboard	33
Figure 8: Monthly Sales Trend Graph	34
Figure 9: Customer Review Display Section	35
Figure 10: System Architecture / Workflow Diagram	36
Figure 11: Review Extraction Process from E-Commerce Platforms	37
Figure 12: Code Implementation Modules	38

CHAPTER 5

INTRODUCTION

In today's digital era, e-commerce platforms such as Amazon, Flipkart, Myntra, and Snapdeal have transformed the way people shop and interact with products. Customers now rely heavily on online reviews and ratings to evaluate products before making purchasing decisions. These reviews provide valuable information regarding product quality, performance, durability, and overall customer satisfaction. At the same time, they serve as an important feedback mechanism for sellers and businesses to improve their products and services.

With the rapid growth of online shopping, the volume of customer reviews has increased significantly. Popular products often receive hundreds or thousands of reviews, making it difficult for users to manually analyze all the feedback. This creates a major challenge for customers who want quick insights and for businesses that need to understand customer opinions efficiently. The unstructured nature of textual reviews further complicates the process of extracting meaningful information.

To address this problem, this project proposes a web-based system that automates the process of collecting and analyzing product reviews from multiple e-commerce platforms. The system uses web scraping techniques to extract real-time reviews and applies Natural Language Processing (NLP) methods to classify them into positive, negative, and neutral sentiments. This helps in converting large amounts of textual data into structured and meaningful insights.

In addition to sentiment classification, the system identifies frequently used keywords in reviews, which helps in highlighting common issues such as product defects, delivery problems, or quality concerns. To enhance user understanding, the system also generates graphical visualizations such as pie charts for sentiment distribution and bar charts for keyword frequency.

A key feature of this project is the estimation of sales impact based on customer sentiment. By analyzing the proportion of negative and neutral reviews, the system predicts how product sales may be affected over time. This provides valuable insights for businesses to take corrective actions and improve customer satisfaction.

The system is implemented using Selenium for web scraping, Flask as the backend framework, and Matplotlib for data visualization. Overall, this project provides an efficient and automated solution for analyzing customer feedback and supports better decision-making for both customers and businesses in the e-commerce ecosystem.

1.ABOUT

This project, titled “**Review Analysis and Sales Impact Prediction System for E-Commerce Platforms,**” is a web-based application designed to extract, analyze, and interpret customer reviews from popular online shopping platforms such as Amazon, Flipkart, Myntra, and Snapdeal.

The primary aim of the project is to automate the process of collecting large volumes of product reviews and converting them into meaningful insights. The system uses web scraping techniques to fetch real-time review data and applies Natural Language Processing (NLP) to analyze the sentiment of customer feedback. Reviews are categorized into positive, negative, and neutral sentiments, enabling users to quickly understand overall product perception.

In addition to sentiment analysis, the system identifies frequently used keywords in customer reviews. These keywords help highlight common product strengths and weaknesses, such as quality, delivery experience, or defects. To enhance usability, the system presents the analysis through graphical visualizations like pie charts and bar graphs.

A unique feature of this project is the prediction of sales impact based on customer sentiment. By evaluating the proportion of negative and neutral reviews, the system estimates how product sales may decline over time, providing valuable business insights.

The project is developed using modern technologies such as Flask for backend development, Selenium for web scraping, and Matplotlib for data visualization. This system provides a smart and efficient solution for both customers and businesses by transforming unstructured review data into actionable insights.

2.OBJECTIVE

The main objectives of the **Review Analysis and Sales Impact Prediction System for E-Commerce Platforms** project are:

1. **Multi-Platform Review Extraction:**
To develop a system that can extract product reviews from multiple e-commerce platforms such as Amazon, Flipkart, Myntra, and Snapdeal in a unified manner.
2. **Automated Sentiment Analysis:**
To analyze customer reviews using Natural Language Processing (NLP) techniques and classify them into positive, negative, and neutral sentiments.
3. **Keyword Identification:**
To identify and track frequently used keywords in customer reviews in order to highlight common product strengths and issues.
4. **Data Visualization:**
To present the analyzed data in a clear and user-friendly format using graphical representations such as pie charts and bar graphs.
5. **Sales Impact Prediction:**
To estimate the effect of customer feedback on product sales by analyzing sentiment distribution and predicting potential sales variation.
6. **Efficient Decision Support:**
To provide meaningful insights that help customers make informed purchasing decisions and assist businesses in improving product quality and performance.

3. KEY FEATURES OF THE REVIEW ANALYSIS AND SALES IMPACT PREDICTION SYSTEM

1. USER INPUT AND PRODUCT HANDLING:

- **Product Input:**
Users can enter a product URL or product name to analyze reviews from different e-commerce platforms.
- **Automatic Link Generation:**
If a product name is provided, the system automatically searches and fetches the product link for analysis.
- **Multi-Platform Support:**
The system supports platforms such as Amazon, Flipkart, Myntra, and Snapdeal.

2. REVIEW EXTRACTION SYSTEM:

- **Real-Time Scraping:**
The system uses web scraping techniques to extract live customer reviews directly from e-commerce websites.
- **Platform Detection:**
It automatically detects the platform based on the product URL and applies the appropriate scraping method.
- **Filtered Data Collection:**
Only relevant and meaningful reviews are collected while removing duplicates and non-English content.

3. SENTIMENT ANALYSIS SYSTEM:

- **Automated Classification:**
Reviews are automatically classified into positive, negative, and neutral categories using NLP techniques.
- **Accurate Analysis:**
The system uses the VADER sentiment analysis model to evaluate the emotional tone of each review.
- **Bulk Processing:**
Large numbers of reviews are processed efficiently in a short time.

4. KEYWORD ANALYSIS SYSTEM:

- **Keyword Identification:**
Frequently used words such as “good”, “bad”, “damaged”, and “quality” are identified from reviews.
- **Insight Generation:**
Keywords help in understanding common customer concerns and product strengths.

5. DATA VISUALIZATION:

- **Sentiment Graphs:**
Pie charts are used to represent the distribution of positive, negative, and neutral reviews.
- **Keyword Charts:**
Bar graphs are generated to show the frequency of important keywords.
- **User-Friendly Display:**
Graphs make it easy for users to quickly understand the analysis results.

6. RATING AND SALES IMPACT ANALYSIS:

- **Dynamic Rating Calculation:**
The system calculates an overall product rating based on sentiment distribution.
- **Sales Impact Prediction:**
It estimates the effect of customer reviews on product sales by analyzing negative and neutral feedback.
- **Performance Insight:**
Users can compare product performance before and after bad reviews.

7. SYSTEM INTERFACE AND USABILITY:

- **Simple Interface:**
The application is designed to be easy to use, even for non-technical users.
- **Fast Processing:**
Results are generated quickly after input is provided.
- **Interactive Output:**
Users can view reviews, graphs, ratings, and insights in one place.

CHAPTER 6

LITERATURE REVIEW

In recent years, the rapid growth of e-commerce platforms has led to an enormous increase in customer-generated data in the form of product reviews and ratings. These reviews play a crucial role in influencing purchasing decisions and shaping product reputation. As a result, many researchers have focused on developing techniques to analyze customer feedback using sentiment analysis and Natural Language Processing (NLP).

Sentiment analysis has become one of the most widely used approaches for understanding customer opinions. Early research in this field primarily focused on classifying text into positive and negative sentiments using basic machine learning algorithms such as Naïve Bayes, Support Vector Machines (SVM), and Decision Trees. These methods required manual feature extraction and were often limited in handling complex language patterns. With the advancement of NLP, lexicon-based approaches such as the VADER (Valence Aware Dictionary and Sentiment Reasoner) model have been developed, which are specifically designed to analyze social media and short text reviews effectively.

Several existing systems have been developed to analyze product reviews on individual platforms. For example, some applications focus solely on Amazon reviews, where they extract customer feedback and perform sentiment classification. However, these systems are limited in scope as they do not support multiple platforms. Other research works have attempted to integrate recommendation systems with sentiment analysis, where user reviews are used to suggest products. While these systems improve user experience, they often lack detailed analytical insights such as keyword extraction and sales impact evaluation.

Web scraping techniques have also been widely used in research to collect data from online platforms. Tools like Selenium and BeautifulSoup enable automated extraction of dynamic web content, making it possible to gather real-time data for analysis. However, scraping from multiple platforms introduces challenges such as varying website structures, dynamic content loading, and anti-bot mechanisms.

Another important area of research is data visualization. Studies have shown that graphical representations such as pie charts and bar graphs significantly improve the understanding of analytical results. Visualization tools help users quickly interpret large datasets and identify patterns in customer feedback without going through raw textual data.

Despite these advancements, most existing systems focus only on sentiment classification and basic analytics. Very few systems attempt to correlate customer sentiment with business outcomes such as product sales. The lack of sales impact prediction limits the practical usability of these systems for business decision-making.

This project addresses these gaps by combining multi-platform review extraction, sentiment analysis, keyword identification, and sales impact prediction into a single integrated system. By doing so, it provides a more comprehensive solution that not only analyzes customer opinions but also predicts their effect on product performance. This makes the system more useful for both customers and businesses in the e-commerce ecosystem.

CHAPTER 7

TECHNOLOGY STACK

A. WEB APPLICATION

1. Frontend Development:

- **HTML, CSS, JavaScript:**

Used for designing and structuring the user interface of the application.

- **Vanilla CSS Design:**

Used to create a clean, responsive, and user-friendly layout for displaying results such as graphs and product details.

2. Backend Development:

- **Python (Flask):**

Used as the backend framework to handle server-side logic, routing, and data processing.

- **Flask Session:**

Used for managing user session data efficiently on the server side.

3. Web Scraping:

- **Selenium:**

Used to extract real-time product reviews, titles, and images from multiple e-commerce platforms.

- **Chrome Driver:**

Acts as a bridge between Selenium and the Chrome browser for automated scraping.

4. Data Processing and Analysis:

- **NLTK (VADER Sentiment Analysis):**

Used to analyze customer reviews and classify them into positive, negative, and neutral sentiments.

- **Custom Keyword Analysis:**

Used to identify frequently occurring words in reviews to highlight product strengths and issues.

5. Data Visualization:

- **Matplotlib:**

Used to generate graphical representations such as pie charts (sentiment distribution) and bar graphs (keyword frequency).

6. Sales Prediction Module:

- **Custom Algorithm (Python-based):**

Used to estimate the impact of customer reviews on product sales based on sentiment ratios.

7.Automation and Background Processing:

- **Threading (Python):**

Used to prefetch product images and improve system performance.

8.Data Handling and Storage:

- **Session-Based Storage:**

Used to temporarily store analysis results during user interaction.

- **File System Storage:**

Used to save generated graphs and images in the static directory.

PROJECT DEVELOPMENT PHASES

A. WEB APPLICATION DEVELOPMENT

1. Phase 1: Requirement Gathering and Analysis (1–2 Weeks):

- Identify system requirements such as extracting product reviews from multiple e-commerce platforms.
- Analyse user needs for quick understanding of product sentiment and performance.
- Define system objectives, scope, and workflow for review analysis and sales prediction.

2. Phase 2: System Design (1–2 Weeks):

- Design overall system architecture including modules like scraping, sentiment analysis, and visualization.
- Plan user interface layout for input, results, and graphs display.
- Prepare flowcharts and data flow diagrams for system working.

3. Phase 3: Development and Integration (3–4 Weeks):

- Develop backend using Python (Flask) for handling requests and processing data.
- Implement web scraping module using Selenium to extract reviews from multiple platforms.
- Integrate sentiment analysis using NLTK (VADER).
- Develop keyword extraction and sales prediction modules.
- Create frontend using HTML, CSS, and JavaScript for displaying results.

4. Phase 4: Visualization and Feature Enhancement (1–2 Weeks):

- Implement graph generation using Matplotlib for sentiment and keyword analysis.
- Add rating calculation and sales impact prediction features.
- Improve user interface for better usability and clarity.

5. Phase 5: Testing and Debugging (1–2 Weeks):

- Test all modules including scraping, analysis, and visualization.
- Fix bugs related to data extraction, incorrect sentiment classification, and UI issues.
- Optimize system performance and ensure accurate results.

CHAPTER 9

DESIGN AND CODE

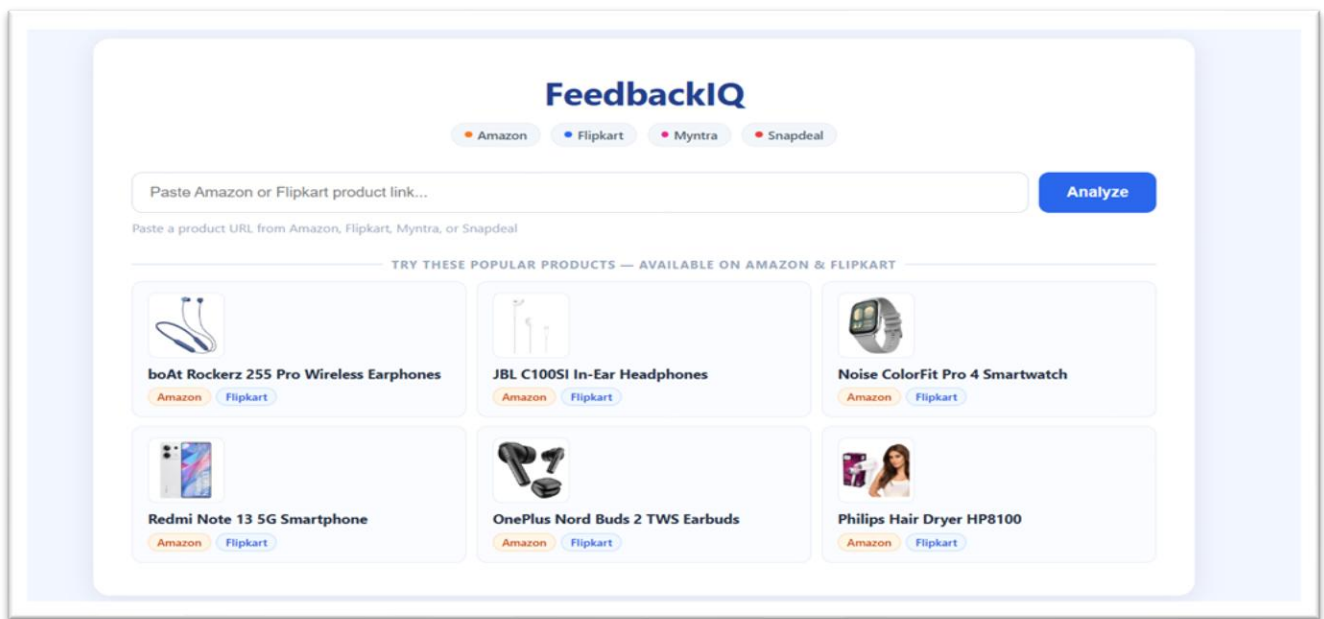


FIGURE 1: LANDING PAGE

FIGURE 2: SEARCH



FIGURE 3: RESULT

SALES IMPACT ANALYSIS



- 3 Sunny**

I recently bought the Goboult Soniq headphones and I'm really impressed with the overall performance. The sound quality is clear with good bass, and it feels balanced whether I'm listening to music, watching videos, or taking calls. The design is sleek and lightweight, so I can wear them for long hours without discomfort. The ear cushions are soft and fit well, which also helps with decent noise isolation. Battery backup is also very good — it easily lasts through long listening sessions without needing frequent charging. Bluetooth connectivity is smooth and stable, and pairing was quick and easy. The controls are simple to use as well. Overall, it's a great value for money product with solid sound, comfort, and battery life. Definitely recommended if you want stylish and reliable headphones at a reasonable price!
- 4 Mohammad Fayyaz**

I recently purchased the GoBoult Soniq and overall I am quite satisfied with the product. The sound quality is really good with clear audio and decent bass, which makes it enjoyable for music and videos. The design looks stylish and feels comfortable to use for long time.
- 5 AAKASH**

I got this headphones yesterday only . so anybody who wants a first impression review , here it is. THE LOOK AND FEEL (9/10) :- The look and feel of this headphone is best , i mean it looks way more expensive at this price point . SOUND QUALITY (9/10) :- The first thing which was impressive was its sound quality . amazing! the bass and overall experience was too good. its a different feeling to listen music on it BUILD QUALITY (7/10) :- Sometimes it feels like its hinge can break if i pull both heads away to wear it so i have to wear it carefully every time FIT AND COMFORT (6/10) :- The main thing which is disappointing is its comfort . it makes the ears pressurized and feels heavy . If someone is buying it for long hours usage , u have to think twice . IT WAS JUST A FIRST IMPRESSION REVIEW . THE COMFORT FACTOR IS HINDERING ME BECAUSE I HAVE USED EARBUDS SINCE LAST 4 YEARS AND NOW I AM SHIFTING TO HEADPHONES . SO IT WILL TAKE SOME TIME TO BE USED TO WITH HEADPHONES

FIGURE 4: SALES IMPACT

CUSTOMER REVIEWS (7 SHOWN)

- 1 Kanhalya prajapati**

I amazed with this headphone. In short words I can say it have a great build quality feels like a very expensive. Sound quality is superb. I liked it's pop mode even I am a bass lover. It's xBass mode increases bass definately but bass is getting harder. I surprised when I was using poweramp equalizer and enabled xBass mode literally this was vibrating on my ears and really, how can I say this was just dancing around my ears and I was literally feeling that. In crazy mode xBass is effective. But only after sometime you will feel irritated due to hard bass in xBass mode. Pop mode has melody experience where chann chann sound bass and overall music experience makes a very sweet, so this mode is made for long music listening session. Pros: - Long Battery Backup - Premium Build Quality - Foldable (neck rest, rotate, folds to carry) every folding feature covered - Soft cushion - Very Good Sound Quality - xBass for crazy mood and short term loud music listening - pop mode is for majority of the time listening - By default getting turned on in pop mode - Sleep and watching experience is good (not excatly as a earbuds but very okay) - Extending more Bass or Treble with poweramp is just handled without distortion even in extreme mode. Cons: - Works perfect with equalizer by default feels lack somewhere. I used poweramp equalizer. - 60ms Gaming mode latency which should not considered because has AUX support with 0ms. - Feels little pressor on head but again comfortable even wearing long 2-3 hour continuously. Overall, this is a great headphone for both casual and intense listening. Definitely worth it! Read more
- 2 @_am_it3**

Overall, the product is good and performs well. The sound quality is clear with decent bass, and the over-ear design is comfortable for long use. Bluetooth connection is stable, and the build feels solid. However, the unit I received came with a scratch/dent, which was disappointing for a brand-new product. It doesn't affect performance, but it definitely affects the first-impression and expectations for new packaging. Apart from the cosmetic issue, the headphones are worth the price. If the quality control is improved, this would easily be a perfect 5-star product.

Code:

File name 1: Scrapper.py

```
import re
import os
import time
import random
import json
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CHROMEDRIVER_PATH = os.path.join(BASE_DIR, "chromedriver.exe")
def _make_driver():
    options = Options()
    options.add_argument("--headless=new")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")
    options.add_argument("--disable-blink-features=AutomationControlled")
    options.add_argument("--window-size=1920,1080")
    options.add_argument("--lang=en-US,en;q=0.9")
    options.add_argument("--disable-infobars")
    options.add_argument("--disable-extensions")
    options.add_experimental_option("excludeSwitches", ["enable-automation"])
    options.add_experimental_option("useAutomationExtension", False)
    options.add_argument(
        "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/134.0.0.0 Safari/537.36"
    )
    service = Service(CHROMEDRIVER_PATH)
    driver = webdriver.Chrome(service=service, options=options)
    # Inject BEFORE any page loads so every navigation is covered
    driver.execute_cdp_cmd("Page.addScriptToEvaluateOnNewDocument", {
        "source": """
Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
Object.defineProperty(navigator, 'plugins', {get: () => [1, 2, 3]});
Object.defineProperty(navigator, 'languages', {get: () => ['en-US', 'en']});
window.chrome = { runtime: {} };
        """
    })
    return driver

def _human_sleep(lo=2.0, hi=4.0):
    time.sleep(random.uniform(lo, hi))

def _is_blocked(driver):
    """Return True if Amazon is showing a CAPTCHA or robot-check page."""
    src = driver.page_source.lower()
    return any(k in src for k in [
        "enter the characters you see below",
        "sorry, we just need to make sure",
        "robot check",
        "captcha",
        "api-services-support@amazon",
    ])

```

—helpers

```
def _detect_platform(url):
    if "amazon." in url: return "amazon"
    if "flipkart." in url: return "flipkart"
    if "myntra." in url: return "myntra"
    if "snapdeal." in url: return "snapdeal"
    return "unknown"
```

```
def _is_english(text):
    if not text:
        return False
    return sum(1 for c in text if ord(c) < 128) / len(text) >= 0.80
```

AMAZON

```
def _scrape_amazon(driver, url):
    domain_m = re.search(r"(https?:\/\/(?:www\.)?amazon\[a-z.\]+)\/", url)
    domain = domain_m.group(1) if domain_m else "https://www.amazon.in"

    # Warm up on homepage first so cookies are set before hitting a product page
    driver.get(domain)
    _human_sleep(2, 3.5)
    driver.get(url)
    _human_sleep(3, 5)

    if _is_blocked(driver):
        return None, None, []
```

Title

title = None

for sel in ["#productTitle", "#title"]:

try:

el = driver.find_element(By.CSS_SELECTOR, sel)

title = el.text.strip()

if title:

break

except Exception:

pass

Image

image_url = None

for sel in ["#landingImage", "#imgBlkFront", "#main-image"]:

try:

el = driver.find_element(By.CSS_SELECTOR, sel)

raw = (el.get_attribute("data-a-dynamic-image")

or el.get_attribute("data-old-hires")

or el.get_attribute("src"))

if raw and raw.startswith("{"):

image_url = list(json.loads(raw).keys())[0]

elif raw:

image_url = raw

if image_url:

break

except Exception:

pass

Extract ASIN for navigating to dedicated review pages

asin_m = re.search(r'/dp/([A-Z0-9]{10})', url)

asin = asin_m.group(1) if asin_m else None

def _collect_page_reviews(seen):

```

found = []

for scroll_y in ["document.body.scrollHeight * 0.4",
                "document.body.scrollHeight * 0.7",
                "document.body.scrollHeight"]:
    driver.execute_script(f"window.scrollTo(0, {scroll_y})")
    _human_sleep(1.2, 2.2)
    for sel in ["li[data-hook='review']", "div[data-hook='review']"]:
        for card in driver.find_elements(By.CSS_SELECTOR, sel):
            try:
                name = card.find_element(
                    By.CSS_SELECTOR, "span.a-profile-name"
                ).text.strip() or "Anonymous"
            except Exception:
                name = "Anonymous"
            try:
                body = card.find_element(
                    By.CSS_SELECTOR, "span[data-hook='review-body']"
                ).text.strip()
                body = re.sub(r'^\s*[\d.]+\s+ out of \d+ stars\s*\s*', "", body).strip().strip()
            except Exception:
                continue
            if body and body not in seen and _is_english(body):
                seen.add(body)
                found.append({"name": name, "text": body})
    return found

```

Product page top reviews (usually positive)

```

seen_texts = set()
reviews = _collect_page_reviews(seen_texts)

# Navigate to dedicated reviews pages for a balanced picture
if asin:
    # all reviews sorted by recency (mixed ratings)
    # then critical-only (1–2 star) to ensure negatives are included
    for filter_param in ["sortBy=recent", "sortBy=recent&filterByStar=critical"]:
        try:
            driver.get(f"{domain}/product-reviews/{asin}?{filter_param}")
            _human_sleep(3, 5)
            if _is_blocked(driver):
                break
            page_revs = _collect_page_reviews(seen_texts)
            reviews.extend(page_revs)
        except Exception:
            pass

return title, image_url, reviews

```

FLIPKART

```

def _flipkart_reviews_url(url):
    clean = url.split("?")[0].rstrip("/")
    m = re.search(r"/p/(itm[A-Za-z0-9]+)", clean)
    if m:
        slug_m = re.search(r"flipkart\.com/([^\s]+)/", clean)
        slug = slug_m.group(1) if slug_m else "product"
        return f"https://www.flipkart.com/{slug}/product-reviews/{m.group(1)}"

```

```

return url

```

```

def _scrape_flipkart(driver, url):
    driver.get(url)
    time.sleep(3)

    title = None

```

```

for sel in ["h1", "span.B_NuCI", "span.VU-ZEz"]:

try:
    el = driver.find_element(By.CSS_SELECTOR, sel)

    t = el.text.strip()
    if t and len(t) > 5:
        title = t
        break
    except Exception:
        pass

image_url = None
try:
    for img in driver.find_elements(By.TAG_NAME, "img"):
        src = img.get_attribute("src") or ""
        if "rukminim" in src and "128" not in src:
            image_url = src
            break
    except Exception:
        pass

reviews_url = _flipkart_reviews_url(url)
driver.get(reviews_url)
time.sleep(4)

raw = driver.execute_script("""
    var results = [];
    var spans = document.querySelectorAll("span.css-1jxf684");

    for (var span of spans) {
        var text = (span.innerText || "").trim();

        if (!text || text.length < 15) continue;
        var container = null, el = span;
        for (var i = 0; i < 12; i++) {
            el = el.parentElement;
            if (!el) break;
            for (var a of el.attributes) {
                if (a.name.startsWith("data-observerid")) { container = el; break; }
            }
            if (container) break;
        }
        var name = "Anonymous";
        if (container) {
            var autoDivs = Array.from(container.querySelectorAll("div[dir='auto']"))
                .map(d => (d.innerText || "").trim()).filter(t => t.length > 0);
            for (var t of autoDivs) {
                if (/^\d+\.\d+$/.test(t)) continue;
                if (t.startsWith("Review for")) continue;
                if (t.startsWith(",")) continue;
                if (t === "\\u2022") continue;
                if (t.length > 60) continue;
                name = t; break;
            }
        }
        results.push({ name: name, text: text });
    }
    return results;
""")

reviews = []
for r in (raw or []):
    text = (r.get("text") or "").strip()

```

```

name = (r.get("name") or "Anonymous").strip() or "Anonymous"
    if text and _is_english(text):
        reviews.append({"name": name, "text": text})

return title, image_url, reviews

# MYNTRA
def _myntra_product_id(url):
    m = re.search(r'/(?d{6,})/buy', url)

return m.group(1) if m else None

def _parse_myntra_cards(driver):
    reviews = []
    seen = set()
    for card in driver.find_elements(By.CSS_SELECTOR, "div.user-review-userReviewWrapper"):
        try:
            text = card.find_element(
                By.CSS_SELECTOR, "div.user-review-reviewTextWrapper"
            ).text.strip()
        except Exception:
            continue

        name = "Anonymous"
        try:
            spans = card.find_elements(By.TAG_NAME, "span")
            non_empty = [s.text.strip() for s in spans if s.text.strip()]
            # non_empty: ["5"(rating), "Reviewer Name", "Date"]
            if len(non_empty) >= 2:
                candidate = non_empty[1]
                if not re.search(r'^\d+${4}\d+ \w+ \d+', candidate):
                    name = candidate
        except Exception:
            pass

        if text and text not in seen and _is_english(text):
            seen.add(text)
            reviews.append({"name": name, "text": text})
    return reviews

def _scrape_myntra(driver, url):
    # Step 1: product page → title + image
    driver.get(url)
    time.sleep(4)

    brand, pname = "", ""
    try:
        brand = driver.find_element(By.CSS_SELECTOR, "h1.pdp-title").text.strip()
    except Exception:
        pass
    try:
        pname = driver.find_element(By.CSS_SELECTOR, "h1.pdp-name").text.strip()
    except Exception:
        pass
    title = f"{brand} {pname}".strip() or None

    image_url = None
    try:
        for img in driver.find_elements(By.TAG_NAME, "img"):

```

```

src = img.get_attribute("src") or ""
    if ("assets.myntassets.com" in src
        and "logo" not in src and "banner" not in src
        and "nav" not in src and "icon" not in src):

src    = re.sub(r'h_\d+', 'h_500', src)
    src    = re.sub(r'w_\d+', 'w_500', src)
    image_url = src
    break
except Exception:

pass

# Step 2: dedicated /reviews/{id} page → 10+ reviews
product_id = _myntra_product_id(url)
if product_id:
    driver.get(f"https://www.myntra.com/reviews/{product_id}")
    time.sleep(4)
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")

time.sleep(2)
reviews = _parse_myntra_cards(driver)
else:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight / 2)")
    time.sleep(2)
    reviews = _parse_myntra_cards(driver)

return title, image_url, reviews
# SNAPDEAL

def _scrape_snapdeal(driver, url):
    driver.get(url)
    time.sleep(5)
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight * 0.6)")
    time.sleep(3)

    # Title
    title = None

for sel in ["h1.pdp-e-i-head", "h1", "span[itemprop='name']"]:
    try:
        el = driver.find_element(By.CSS_SELECTOR, sel)
        t = el.text.strip()
        if t and len(t) > 5:
            title = t
            break
    except Exception:
        pass

# Image
image_url = None
for sel in ["img#main-product-image", "img.cloudzoom", "div.product-thumbnails img"]:
    try:
        el = driver.find_element(By.CSS_SELECTOR, sel)
        src = (el.get_attribute("src")
              or el.get_attribute("data-src")
              or "")
    except Exception:
        pass

if src:
    image_url = src
    break
except Exception:
    pass

# Reviews

```



```

reviews = []
for sel in ["div.user-review", "div.review-description", "p.review-description"]:
    cards = driver.find_elements(By.CSS_SELECTOR, sel)
    if cards:
        for card in cards:
            text = card.text.strip()

# Try to get name from sibling/parent
    name = "Anonymous"
    try:
        name_el = card.find_element(By.CSS_SELECTOR,
                                    ".reviewer-name, .review-author, span.name")
        name = name_el.text.strip() or "Anonymous"

except Exception:
    pass
    if text and len(text) > 15 and _is_english(text):
        reviews.append({"name": name, "text": text})
    break

return title, image_url, reviews
# PUBLIC API

def get_product_link(search_url):
    """Amazon keyword search → first product URL."""
    driver = _make_driver()
    try:
        driver.get(search_url)
        time.sleep(2)
        for link in driver.find_elements(By.CSS_SELECTOR, "a.a-link-normal.s-no-outline"):
            href = link.get_attribute("href")

    if href and "/dp/" in href:
        return href.split("?")[0]
    except Exception:
        pass
    finally:
        driver.quit()
    return None

def scrape_product(url):
    """
    Single browser session. Returns:
    { "title": str|None, "image": str|None, "reviews": [{"name":..., "text":...}] }
    """
    driver = _make_driver()
    try:
        platform = _detect_platform(url)
        if platform == "amazon":
            title, image, reviews = _scrape_amazon(driver, url)
        elif platform == "flipkart":
            title, image, reviews = _scrape_flipkart(driver, url)
        elif platform == "myntra":
            title, image, reviews = _scrape_myntra(driver, url)
        elif platform == "snapdeal":
            title, image, reviews = _scrape_snapdeal(driver, url)
        else:
            return {"title": None, "image": None, "reviews": [],
                    "error": "Unsupported platform. Paste a link from Amazon, Flipkart, Myntra, or Snapdeal."}
        return {"title": title, "image": image, "reviews": reviews, "platform": platform}
    except Exception:
        return {"title": None, "image": None, "reviews": [], "platform": "unknown"}
    finally: driver.quit()

```

File Name 2: Analyzer.py

```
import re

from collections import Counter
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
from nltk.corpus import stopwords

nltk.download('vader_lexicon', quiet=True)
nltk.download('stopwords', quiet=True)

sia = SentimentIntensityAnalyzer()

_STOP = set(stopwords.words('english')) | {
    'product', 'item', 'one', 'get', 'got', 'use', 'used', 'using',
    'would', 'could', 'also', 'like', 'really', 'very', 'much',
    'even', 'still', 'well', 'just', 'buy', 'bought', 'ordered',
    'order', 'delivery', 'delivered', 'amazon', 'flipkart', 'seller',
    'price', 'money', 'review', 'reviews', 'recommend', 'star', 'stars',
    'good', 'great', 'nice', 'bad', 'okay', 'bit', 'lot',
}

KEYWORDS = [
    "good", "bad", "poor",
    "excellent", "worst", "damaged",
    "broken", "quality", "fast",
    "slow", "fake", "original",
    "value", "waste", "perfect",
]

def _extract_words(text):
    words = re.findall(r'\b[a-z]{3,}\b', text.lower())
    return [w for w in words if w not in _STOP]

def _top_keywords(word_lists, n=8):
    counter = Counter()
    for words in word_lists:
        counter.update(words)
    return [word for word, _ in counter.most_common(n)]

def analyze_reviews(reviews):
```

```

result = {
    "positive": 0,
    "negative": 0,
    "neutral": 0,
    "keywords": {k: 0 for k in KEYWORDS},
    "sentiment_keywords": {"positive": [], "negative": [], "neutral": []},
}

pos_words, neg_words, neu_words = [], [], []

for review in reviews:
    review_lower = review.lower()
    score = sia.polarity_scores(review)
    words = _extract_words(review)

    if score['compound'] >= 0.05:
        result["positive"] += 1
        pos_words.append(words)
    elif score['compound'] <= -0.05:
        result["negative"] += 1
        neg_words.append(words)
    else:
        result["neutral"] += 1
        neu_words.append(words)

    for key in KEYWORDS:
        if key in review_lower:
            result["keywords"][key] += 1

result["keywords"] = {k: v for k, v in result["keywords"].items() if v > 0}
result["sentiment_keywords"] = {
    "positive": _top_keywords(pos_words),

    "negative": _top_keywords(neg_words),
    "neutral": _top_keywords(neu_words),
}

return result

```

File Name 3: Sales.py

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

```

— Model

"Before bad reviews" = peak launch potential (product always starts

higher — hype, early-adopter boost, no negative signal yet).

#

"After bad reviews" = current state reduced by:

• negative reviews → heavy drag ($\times 0.75$ per neg-ratio point)

• neutral reviews → light drag ($\times 0.12$ per neu-ratio point)

• natural product-age decay (minimum 5 % always shown)

This guarantees before > after even for fully-positive products,

which is realistic — no product sustains its launch peak forever.

_LAUNCH_BOOST = 1.30 # peak is always 30 % above the settled baseline

_MIN_DROP = 0.05 # at least 5 % natural decay even with zero bad reviews

def estimate_sales_impact(result, rating):

total = result["positive"] + result["negative"] + result["neutral"]

if total == 0:

return None

pos_ratio = result["positive"] / total

neg_ratio = result["negative"] / total

neu_ratio = result["neutral"] / total

Settled baseline driven by rating + positive share

rating_factor = 0.4 + (rating / 5) * 0.6 # 0.40 – 1.00

base = 1000 * rating_factor * (0.5 + pos_ratio) # ~200 – 1500

sales_before = round(base * _LAUNCH_BOOST)

neg_drag = neg_ratio * 0.75

neu_drag = neu_ratio * 0.12

total_drag = max(_MIN_DROP, neg_drag + neu_drag) # floor at 5 %

sales_after = round(sales_before * (1 - total_drag))

drop_pct = round((sales_before - sales_after) / sales_before * 100, 1)

if drop_pct < 10:

severity = "Low"

elif drop_pct < 30:

severity = "Moderate"

elif drop_pct < 55:

severity = "High"

else:

severity = "Critical"

```

return {
    "sales_before": sales_before,
    "sales_after": sales_after,
    "drop_pct": drop_pct,
    "neg_pct": round(neg_ratio * 100, 1),
    "pos_pct": round(pos_ratio * 100, 1),
    "severity": severity,
}

```

— Chart

```

def generate_sales_chart(impact):
    before = impact["sales_before"]
    after = impact["sales_after"]
    drop = impact["drop_pct"]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))
    fig.patch.set_facecolor("#f8faff")

    # — Panel 1 : Bar comparison
    bars = ax1.bar(
        ["Before\nBad Reviews", "After\nBad Reviews"],
        [before, after],
        color=["#2563eb", "#ef4444"],
        width=0.45,
        edgecolor="white",
        linewidth=1.5,
        zorder=3,
    )
    ax1.set_facecolor("#f8faff")
    ax1.yaxis.grid(True, color="#e2e8f0", linewidth=0.8, zorder=0)
    ax1.set_axisbelow(True)
    ax1.spines["top"].set_visible(False)
    ax1.spines["right"].set_visible(False)
    ax1.spines["left"].set_color("#e2e8f0")
    ax1.spines["bottom"].set_color("#e2e8f0")
    ax1.set_ylabel("Sales Index (units)", fontsize=9, color="#475569")
    ax1.set_title("Sales Impact: Before vs After", fontsize=11,
                  fontweight="bold", color="#1e293b", pad=10)
    ax1.tick_params(colors="#475569", labels=9)

    # Value label inside "Before" bar
    ax1.text(

```

```

bars[0].get_x() + bars[0].get_width() / 2,
    bars[0].get_height() * 0.5,
    f"{before:;}",
    ha="center", va="center",
    fontsize=11, fontweight="bold", color="white",
)

# Value + drop label inside "After" bar
ax1.text(
    bars[1].get_x() + bars[1].get_width() / 2,
    bars[1].get_height() * 0.5,
    f"{after:;}\n ▼ {drop}% drop",
    ha="center", va="center",
    fontsize=10, fontweight="bold", color="white",

linespacing=1.6,
)

# — Panel 2 : Monthly trend line
months = np.arange(1, 13)
ramp = np.linspace(before * 0.55, before, 6)
decay = np.linspace(before, after, 6)
sales = np.concatenate([ramp, decay])

ax2.set_facecolor("#f8faff")
ax2.fill_between(months[:6], sales[:6], alpha=0.13, color="#2563eb")
ax2.fill_between(months[5:], sales[5:], alpha=0.10, color="#ef4444")
ax2.plot(months[:6], sales[:6], color="#2563eb", linewidth=2.2,
        label="Growth phase")
ax2.plot(months[5:], sales[5:], color="#ef4444", linewidth=2.2,
        linestyle="--", label="Post bad-review decline")

# Inflection marker (placed at top-left of right half to avoid overlap)
ax2.axvline(x=6, color="#f59e0b", linewidth=1.4, linestyle=":")
ax2.text(6.2, sales.min() + (sales.max() - sales.min()) * 0.15,
        "Bad reviews\ninflection",
        fontsize=7.5, color="#92400e", va="bottom")

ax2.yaxis.grid(True, color="#e2e8f0", linewidth=0.8, zorder=0)
ax2.set_axisbelow(True)
ax2.spines["top"].set_visible(False)
ax2.spines["right"].set_visible(False)
ax2.spines["left"].set_color("#e2e8f0")

```

```

ax2.spines["bottom"].set_color("#e2e8f0")
ax2.set_xlabel("Month", fontsize=9, color="#475569")
ax2.set_ylabel("Sales Index (units)", fontsize=9, color="#475569")
ax2.set_title("Simulated Monthly Sales Trend", fontsize=11,
              fontweight="bold", color="#1e293b", pad=10)
ax2.set_xticks(months)
ax2.tick_params(colors="#475569", labels=8)
ax2.legend(fontsize=8, frameon=False, loc="lower left")

plt.tight_layout(pad=2.0)
plt.savefig("static/sales_impact.png", bbox_inches="tight", dpi=120,
           facecolor="#f8faff")
plt.close()

```

File Name 4: app.py

```

from flask import Flask, render_template, request, redirect, url_for, session, jsonify
from flask_session import Session
from analyser import analyze_reviews
from scraper import scrape_product, get_product_link, _make_driver
from sales import estimate_sales_impact, generate_sales_chart
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import os
import threading
import urllib.parse
import secrets
from selenium.webdriver.common.by import By

app = Flask(__name__)
app.secret_key = secrets.token_hex(16)

# — Server-side filesystem session (no 4KB cookie limit)
app.config["SESSION_TYPE"] = "filesystem"
app.config["SESSION_FILE_DIR"] = os.path.join(os.path.dirname(__file__), "flask_sessions")
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_USE_SIGNER"] = True
os.makedirs(app.config["SESSION_FILE_DIR"], exist_ok=True)
Session(app)

if not os.path.exists("static"):
    os.makedirs("static")

```

— Featured product thumbnail prefetch

```
FEATURED_PRODUCTS = [
    "boAt Rockerz 255 Pro Wireless Earphones",
    "JBL C100SI In-Ear Headphones",
    "Noise ColorFit Pro 4 Smartwatch",
    "Redmi Note 13 5G Smartphone",
    "Oneplus Nord Buds 2 TWS Earbuds",
    "Philips Hair Dryer HP8100"]

_thumb_cache: dict = {} # product name → image URL

def _prefetch_images():
    driver = _make_driver()
    try:
        for name in FEATURED_PRODUCTS:
            if name in _thumb_cache:
                continue
            try:
                query = urllib.parse.quote(name)
                driver.get(f"https://www.amazon.in/s?k={query}")
                import time; time.sleep(2)
                for img in driver.find_elements(By.CSS_SELECTOR, "img.s-image"):
                    src = img.get_attribute("src") or ""
                    if src.startswith("http"):
                        _thumb_cache[name] = src
                        break
            except Exception:
                pass
    finally:
        driver.quit()
threading.Thread(target=_prefetch_images, daemon=True).start()

@app.route("/api/product-image")
def product_image_api():
    name = request.args.get("q", "").strip()
    return jsonify({"url": _thumb_cache.get(name)})
```

— Graphs

```
def generate_graphs(result):
    all_labels = ['Positive', 'Negative', 'Neutral']
    all_values = [result['positive'], result['negative'], result['neutral']]
    all_colors = ['#4ade80', '#f87171', '#94a3b8']
```



```

total_val = sum(all_values) if sum(all_values) > 0 else 1

# Use a tiny floor so zero-value categories still render as a visible sliver
plot_values = [max(v, 0.01) for v in all_values] if sum(all_values) > 0 else [1, 0.01, 0.01]
labels_with_pct = [f"{l} ({v/total_val*100:.1f}%)" for l, v in zip(all_labels, all_values)]

_, ax = plt.subplots(figsize=(6.5, 3.5))
wedges, _ = ax.pie(
    plot_values,
    colors=all_colors,

startangle=90,
    wedgeprops=dict(linewidth=1.5, edgecolor='white'),
)

legend = ax.legend(
    wedges,
    labels_with_pct,
    title="Distribution",
    loc="center left",
    bbox_to_anchor=(0.95, 0.5),
    frameon=False,
    fontsize=10
)
legend.get_title().set_fontweight('bold')
legend.get_title().set_fontsize(11)

ax.set_title("Sentiment Analysis", fontsize=13, fontweight='bold', pad=14)
plt.savefig("static/sentiment.png", bbox_inches="tight", dpi=120)
plt.close()

keywords = {k: v for k, v in result['keywords'].items() if v > 0}
_, ax = plt.subplots(figsize=(6.5, 3.5))
if keywords:
    keys = list(keywords.keys())
    counts = list(keywords.values())
    bars = ax.bar(keys, counts, color='#2563eb', width=0.6, edgecolor='white')
    for bar, count in zip(bars, counts):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.05,
            str(count), ha='center', va='bottom', fontsize=9,
            fontweight='bold', color='#1e293b')
    ax.set_ylabel("Mentions", fontsize=10)
    ax.set_title("Top Keywords in Reviews", fontsize=13, fontweight='bold')
    ax.set_ylim(0, max(counts) * 1.25)

```

```

ax.spines['top'].set_visible(False)

ax.spines['right'].set_visible(False)
plt.xticks(rotation=35, ha='right', fontsize=9)
else:
    ax.text(0.5, 0.5, 'No keywords found', ha='center', va='center',
           transform=ax.transAxes, color='#94a3b8', fontsize=13)
    ax.axis('off')
plt.tight_layout()
plt.savefig("static/keywords.png", bbox_inches="tight", dpi=120)
plt.close()

def calculate_rating(result):
    total = result['positive'] + result['negative'] + result['neutral']
    if total == 0:
        return 0
    score = (result['positive'] * 5 + result['neutral'] * 3 + result['negative'] * 1) / total
    return round(score, 1)

# — Routes

@app.route("/", methods=["GET"])
def home():
    data = session.pop("result_data", None)
    if data:
        return render_template("index.html", **data)
    return render_template("index.html", result=None)

@app.route("/analyze", methods=["POST"])
def analyze():
    product_input = request.form.get("product", "").strip()

    if not product_input:
        session["result_data"] = {"result": None, "error": "Please enter a product URL."}
        return redirect(url_for("home"))

PLATFORM_DOMAINS = ("amazon.", "flipkart.", "mynta.", "snapdeal.")

if product_input.startswith("http://") or product_input.startswith("https://"):
    product_url = product_input
elif any(d in product_input for d in PLATFORM_DOMAINS):
    # URL pasted without protocol prefix — add https://

```

```
product_url = "https://" + product_input.lstrip("/")
```

```
else:
```

```
    query = urllib.parse.quote(product_input)
```

```
    product_url = get_product_link(f"https://www.amazon.in/s?k={query}")
```

```
if not product_url:
```

```
    session["result_data"] = {
```

```
        "result": None,
```

```
        "error": "Could not find a product. Try pasting the product link directly."
```

```
    }
```

```
    return redirect(url_for("home"))
```

```
data = scrape_product(product_url)
```

```
if data.get("error"):
```

```
    session["result_data"] = {"result": None, "error": data["error"]}
```

```
    return redirect(url_for("home"))
```

```
if not data["reviews"]:
```

```
    platform = data.get("platform", "unknown")
```

```
    if platform == "amazon":
```

```
        msg = ("Amazon blocked the request (bot protection). "
```

```
            "Wait 30–60 seconds and try again, or paste a Flipkart link for the same product.")
```

```
    else:
```

```
        msg = "No reviews found. The site may have blocked the request, or this product has no English reviews yet."
```

```
    session["result_data"] = {"result": None, "error": msg}
```

```
    return redirect(url_for("home"))
```

```
reviews = data["reviews"]
```

```
display_name = data["title"] or product_input
```

```
product_image = data["image"]
```

```
texts = [r["text"] for r in reviews]
```

```
result = analyze_reviews(texts)
```

```
generate_graphs(result)
```

```
rating = calculate_rating(result)
```

```
impact = estimate_sales_impact(result, rating)
```

```
if impact:
```

```
    generate_sales_chart(impact)
```

```
session["result_data"] = {
```

```
    "result": result,
```

```
    "reviews": reviews[:10],
```

```

        "rating":    rating,

"product_name": display_name,
    "product_image": product_image,
    "total_reviews": len(reviews),
    "impact":    impact,
    "platform":    data.get("platform", "unknown"),
}

return redirect(url_for("home"))

if __name__ == "__main__":
    app.run(debug=True, port=5005)

```

File Name 5: index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>FeedbackIQ</title>
<style>
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #f0f4ff;
    color: #1e293b;
    min-height: 100vh;
}

.container {
    width: 68%;
    margin: 44px auto;
    background: white;
    padding: 36px;
    border-radius: 16px;
    box-shadow: 0 4px 28px rgba(0,0,0,0.09);
}

/* — Header — */
.header { text-align: center; margin-bottom: 28px; }

.header h1 {
    color: #1e3a8a;

```

```

    font-size: 2.2rem;

letter-spacing: -0.5px;
}

.badges {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin-top: 10px;
    flex-wrap: wrap;
}

.badge {
    display: inline-flex;
    align-items: center;
    gap: 5px;
    background: #f1f5f9;
    border: 1px solid #e2e8f0;
    border-radius: 20px;
    padding: 4px 12px;
    font-size: 0.78rem;
    color: #475569;
    font-weight: 500;
}

.badge-dot {
    width: 7px; height: 7px;
    border-radius: 50%;
}

.badge-dot.amazon { background: #f97316; }
.badge-dot.flipkart { background: #2563eb; }
.badge-dot.myntra { background: #e91e8c; }
.badge-dot.snapdeal { background: #e53935; }

/* — Search — */

.search-bar { display: flex; gap: 10px; }

.search-bar input {
    flex: 1;
    padding: 13px 16px;
    border: 1.5px solid #cbd5e1;
    border-radius: 10px;

```

```

    font-size: 15px;

outline: none;
    transition: border-color 0.2s;
}

.search-bar input:focus { border-color: #2563eb; }

.search-bar button {
    padding: 13px 26px;
    background: #2563eb;
    color: white;
    border: none;
    border-radius: 10px;
    font-size: 15px;
    cursor: pointer;
    font-weight: 700;
    letter-spacing: 0.2px;
    transition: background 0.2s;
}

.search-bar button:hover { background: #1d4ed8; }

.hint {
    margin-top: 8px;
    font-size: 0.8rem;
    color: #94a3b8;
}

.error {
    color: #dc2626;
    background: #fef2f2;
    border: 1px solid #fecaca;
    border-radius: 8px;
    padding: 10px 14px;
    margin-top: 16px;
    font-size: 14px;
}

/* — Featured Products — */
.featured-section {
    margin-top: 24px;
}

```

```

.featured-label {

font-size: 0.75rem;
  font-weight: 700;
  color: #94a3b8;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  margin-bottom: 10px;
  display: flex;
  align-items: center;
  gap: 8px;
}

.featured-label::before,
.featured-label::after {
  content: "";
  flex: 1;
  height: 1px;
  background: #e2e8f0;
}

.featured-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.product-card {
  background: #f8faff;
  border: 1.5px solid #e2e8f0;
  border-radius: 10px;
  padding: 12px 14px;
  cursor: pointer;
  transition: border-color 0.18s, box-shadow 0.18s, background 0.18s;
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.product-card:hover {
  border-color: #2563eb;
  background: #eff6ff;
  box-shadow: 0 2px 10px rgba(37,99,235,0.10);
}

```

```
.product-card-name {  
  font-size: 0.88rem;  
  font-weight: 700;  
  color: #1e293b;  
  line-height: 1.35;  
}
```

```
.product-card-platforms {  
  display: flex;  
  gap: 5px;  
  flex-wrap: wrap;  
}
```

```
.platform-pill {  
  font-size: 0.68rem;  
  font-weight: 600;  
  padding: 2px 8px;  
  border-radius: 20px;  
  letter-spacing: 0.02em;  
}
```

```
.platform-pill.amazon { background: #fff4e6; color: #c2410c; border: 1px solid #fed7aa; }  
.platform-pill.flipkart { background: #eff6ff; color: #1d4ed8; border: 1px solid #bfdbfe; }
```

```
.product-card-icon {  
  font-size: 2rem;  
  line-height: 1;  
  margin-bottom: 2px;  
}
```

```
.product-card-img {  
  width: 72px;  
  height: 72px;  
  object-fit: contain;  
  border-radius: 8px;  
  background: white;  
  border: 1px solid #e2e8f0;  
  padding: 4px;  
  display: none;  
  margin-bottom: 2px;  
}
```



```
/* Platform source badge on result */
```

```
.source-badge {  
  display: inline-flex;  
  align-items: center;  
  gap: 5px;  
  padding: 3px 11px;  
  border-radius: 20px;  
  font-size: 0.75rem;  
  font-weight: 700;  
  margin-left: 8px;  
  vertical-align: middle;  
}  
  
.source-badge.amazon { background: #fff4e6; color: #c2410c; border: 1px solid #fed7aa; }  
.source-badge.flipkart { background: #fff6ff; color: #1d4ed8; border: 1px solid #bfdbfe; }  
.source-badge.myntra { background: #fdf2f8; color: #9d174d; border: 1px solid #fbcfe8; }  
.source-badge.snapdeal { background: #fef2f2; color: #b91c1c; border: 1px solid #fecaca; }
```

```
/* — Product Header — */
```

```
.product-header {  
  display: flex;  
  align-items: center;  
  gap: 20px;  
  margin-top: 30px;  
  padding: 18px;  
  background: #f8faff;  
  border-radius: 12px;  
  border: 1px solid #e2e8f0;  
}
```

```
.product-header img {  
  width: 100px;  
  height: 100px;  
  object-fit: contain;  
  border-radius: 8px;  
  background: white;  
  border: 1px solid #e2e8f0;  
  padding: 6px;  
  flex-shrink: 0;  
}
```

```
.product-meta h2 {  
  font-size: 1.05rem;  
  color: #1e293b;
```

```

    margin-bottom: 8px;

    line-height: 1.4;
}

.rating-badge {
    display: inline-block;
    background: #fef3c7;
    color: #92400e;
    padding: 4px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 0.95rem;
}

/* — Sentiment — */
.sentiment-grid {
    display: flex;
    gap: 14px;
    margin-top: 22px;
}

.sentiment-box {
    flex: 1;
    padding: 16px;
    border-radius: 12px;
    text-align: center;
    font-weight: 600;
    font-size: 0.95rem;
}

.sentiment-box.positive { background: #dcfce7; color: #166534; }
.sentiment-box.negative { background: #fee2e2; color: #991b1b; }
.sentiment-box.neutral { background: #fff2cc; color: #8d6e14; }

.sentiment-box span {
    display: block;
    font-size: 1.8rem;
    font-weight: 800;
    margin-top: 4px;
}

/* — Graphs — */
.graphs {

```

```

    display: flex;

gap: 16px;
    margin-top: 22px;
}

.graphs img {
    flex: 1;
    width: 48%;
    max-height: 280px;
    object-fit: contain;
    border-radius: 10px;
    border: 1px solid #e2e8f0;
    background: #f8faff;
}

/* — Sentiment Keywords — */
.kw-section {
    margin-top: 20px;
    padding: 18px 20px;
    background: #f8faff;
    border: 1px solid #e2e8f0;
    border-radius: 12px;
}

.kw-section-title {
    font-size: 0.75rem;
    font-weight: 700;
    color: #94a3b8;
    text-transform: uppercase;
    letter-spacing: 0.08em;
    margin-bottom: 14px;
    display: flex;
    align-items: center;
    gap: 8px;
}

.kw-section-title::before,
.kw-section-title::after {
    content: "";
    flex: 1;
    height: 1px;
    background: #e2e8f0;
}

```

```

.kw-grid {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
}

.kw-group {
  flex: 1;
  min-width: 160px;
  padding: 14px 16px;
  border-radius: 10px;
}

.kw-group.positive {
  background: #f0fdf4;
  border-color: #bbf7d0;
}

.kw-group.negative {
  background: #fef2f2;
  border-color: #fecaca;
}

.kw-group.neutral {
  background: #fff5f5;
  border-color: #ffe8e8;
}

.kw-group-label {
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  margin-bottom: 10px;
  display: flex;
  align-items: center;
  gap: 6px;
}

.kw-group.positive .kw-group-label { color: #15803d; }
.kw-group.negative .kw-group-label { color: #b91c1c; }
.kw-group.neutral .kw-group-label { color: #475569; }

```

```

.kw-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  flex-shrink: 0;
}

.kw-group.positive .kw-dot { background: #4ade80; }
.kw-group.negative .kw-dot { background: #f87171; }
.kw-group.neutral .kw-dot { background: #94a3b8; }

.kw-tags {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}

.kw-tag {
  padding: 3px 10px;
  border-radius: 20px;
  font-size: 0.78rem;
  font-weight: 600;
  letter-spacing: 0.01em;
}

.kw-group.positive .kw-tag { background: #dcfce7; color: #166534; border: 1px solid #86efac; }
.kw-group.negative .kw-tag { background: #fee2e2; color: #991b1b; border: 1px solid #fca5a5; }
.kw-group.neutral .kw-tag { background: #e2e8f0; color: #334155; border: 1px solid #cbd5e1; }

.kw-empty {
  font-size: 0.8rem;
  color: #94a3b8;
  font-style: italic;
}

@media (max-width: 640px) {
  .kw-grid { flex-direction: column; }
}

/* — Sales Impact — */
.sales-section { margin-top: 30px; }

```

```

.sales-section h3 {

font-size: 1rem;
color: #1e3a8a;
font-weight: 700;
margin-bottom: 14px;
padding-bottom: 10px;
border-bottom: 2px solid #e2e8f0;
text-transform: uppercase;
letter-spacing: 0.05em;
}

.sales-metrics {
display: flex;
gap: 14px;
margin-bottom: 16px;
flex-wrap: wrap;
}

.sales-card {
flex: 1;
min-width: 130px;
padding: 16px;
border-radius: 12px;
text-align: center;
font-weight: 600;
font-size: 0.85rem;
border: 1px solid #e2e8f0;
}

.sales-card.before { background: #eff6ff; color: #1e40af; }
.sales-card.after { background: #fef2f2; color: #991b1b; }
.sales-card.drop { background: #fff7ed; color: #92400e; }
.sales-card.verdict { background: #f0fdf4; color: #166534; }
.sales-card.verdict.high { background: #fff7ed; color: #92400e; }
.sales-card.verdict.critical { background: #fef2f2; color: #991b1b; }

.sales-card span {
display: block;
font-size: 1.7rem;
font-weight: 800;
margin-top: 4px;
}

```

```

.sales-chart img {

width: 100%;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  background: #f8faff;
}

/* — Reviews — */
.reviews-section { margin-top: 30px; }

.reviews-section h3 {
  font-size: 1rem;
  color: #1e3a8a;
  font-weight: 700;
  margin-bottom: 14px;
  padding-bottom: 10px;
  border-bottom: 2px solid #e2e8f0;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.review-card {
  display: flex;
  gap: 14px;
  align-items: flex-start;
  background: #f8faff;
  border: 1px solid #e2e8f0;
  padding: 14px 16px;
  border-radius: 10px;
  margin-bottom: 10px;
}

.review-number {
  min-width: 28px;
  height: 28px;
  background: #2563eb;
  color: white;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 700;
  font-size: 0.82rem;
}

```

```

    flex-shrink: 0;

margin-top: 1px;
}

.review-content { display: flex; flex-direction: column; gap: 4px; flex: 1; }

.review-name {
    font-weight: 700;
    font-size: 0.85rem;
    color: #1e3a8a;
}

.review-text {
    font-size: 0.93rem;
    line-height: 1.65;
    color: #334155;
}
</style>
</head>
<body>

<div class="container">

    <div class="header">
        <h1>FeedbackIQ</h1>
        <div class="badges">
            <span class="badge"><span class="badge-dot amazon"></span>Amazon</span>
            <span class="badge"><span class="badge-dot flipkart"></span>Flipkart</span>
            <span class="badge"><span class="badge-dot myntra"></span>Myntra</span>
            <span class="badge"><span class="badge-dot snapdeal"></span>Snapdeal</span>
        </div>
    </div>

    <form method="POST" action="/analyze" id="searchForm">
        <div class="search-bar">
            <input type="text" name="product" id="productInput"
                placeholder="Paste Amazon or Flipkart product link..."
                value="{{ request.form.get('product', '') }}">
            <button type="submit">Analyze</button>
        </div>
        <p class="hint">Paste a product URL from Amazon, Flipkart, Myntra, or Snapdeal</p>
    </form>

```


{% if not result %}

```
<div class="featured-section">
  <div class="featured-label">Try these popular products — available on Amazon & Flipkart</div>
  <div class="featured-grid">
    <div class="product-card" onclick="searchProduct('boAt Rockerz 255 Pro Wireless Earphones')">
      <img class="product-card-img" data-product="boAt Rockerz 255 Pro Wireless Earphones" alt="boAt
Rockerz 255">
      <div class="product-card-icon">🎧</div>
      <div class="product-card-name">boAt Rockerz 255 Pro Wireless Earphones</div>
      <div class="product-card-platforms">
        <span class="platform-pill amazon">Amazon</span>
        <span class="platform-pill flipkart">Flipkart</span>
      </div>
    </div>
    <div class="product-card" onclick="searchProduct('JBL C100SI In-Ear Headphones')">
      <img class="product-card-img" data-product="JBL C100SI In-Ear Headphones" alt="JBL C100SI">
      <div class="product-card-icon">🎧</div>
      <div class="product-card-name">JBL C100SI In-Ear Headphones</div>
      <div class="product-card-platforms">
        <span class="platform-pill amazon">Amazon</span>
        <span class="platform-pill flipkart">Flipkart</span>
      </div>
    </div>
    <div class="product-card" onclick="searchProduct('Noise ColorFit Pro 4 Smartwatch')">
      <img class="product-card-img" data-product="Noise ColorFit Pro 4 Smartwatch" alt="Noise ColorFit Pro
4">
      <div class="product-card-icon">🕒</div>
      <div class="product-card-name">Noise ColorFit Pro 4 Smartwatch</div>
      <div class="product-card-platforms">
        <span class="platform-pill amazon">Amazon</span>
        <span class="platform-pill flipkart">Flipkart</span>
      </div>
    </div>
    <div class="product-card" onclick="searchProduct('Redmi Note 13 5G Smartphone')">
      <img class="product-card-img" data-product="Redmi Note 13 5G Smartphone" alt="Redmi Note 13 5G">
      <div class="product-card-icon">📱</div>
      <div class="product-card-name">Redmi Note 13 5G Smartphone</div>
      <div class="product-card-platforms">
        <span class="platform-pill amazon">Amazon</span>
        <span class="platform-pill flipkart">Flipkart</span>
      </div>
    </div>
    <div class="product-card" onclick="searchProduct('Oneplus Nord Buds 2 TWS Earbuds')">
```

```
<img class="product-card-img" data-product="Oneplus Nord Buds 2 TWS Earbuds" alt="OnePlus Nord
```

```
Buds 2">
```

```
<div class="product-card-icon">🎧 </div>
```

```
<div class="product-card-name">OnePlus Nord Buds 2 TWS Earbuds</div>
```

```
<div class="product-card-platforms">
```

```
  <span class="platform-pill amazon">Amazon</span>
```

```
  <span class="platform-pill flipkart">Flipkart</span>
```

```
</div>
```

```
</div>
```

```
<div class="product-card" onclick="searchProduct('Philips Hair Dryer HP8100')">
```

```
  <img class="product-card-img" data-product="Philips Hair Dryer HP8100" alt="Philips Hair Dryer">
```

```
  <div class="product-card-icon">🔌 </div>
```

```
  <div class="product-card-name">Philips Hair Dryer HP8100</div>
```

```
  <div class="product-card-platforms">
```

```
    <span class="platform-pill amazon">Amazon</span>
```

```
    <span class="platform-pill flipkart">Flipkart</span>
```

```
  </div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
<script>
```

```
function searchProduct(name) {
```

```
  document.getElementById('productInput').value = name;
```

```
  document.getElementById('searchForm').submit();
```

```
}
```

```
function loadThumbs() {
```

```
  var imgs = document.querySelectorAll('.product-card-img:not([data-loaded]));
```

```
  if (!imgs.length) return;
```

```
  imgs.forEach(function(img) {
```

```
    fetch('/api/product-image?q=' + encodeURIComponent(img.dataset.product))
```

```
    .then(function(r) { return r.json(); })
```

```
    .then(function(d) {
```

```
      if (d.url) {
```

```
        img.src = d.url;
```

```
        img.style.display = 'block';
```

```
        img.dataset.loaded = '1';
```

```
        img.nextElementSibling.style.display = 'none'; // hide emoji
```

```
      }
```

```
    })
```

```
    .catch(function() {});
```

```
});
```

```
// retry every 4s until all images are loaded
```

```
    if (document.querySelectorAll('.product-card-img:not([data-loaded])').length) {  
        setTimeout(loadThumbs, 4000);  
    }  
}  
loadThumbs();  
</script>
```

```
{% if error %}  
<div class="error">{{ error }}</div>  
{% endif %}
```

```
{% if result %}
```

```
<div class="product-header">  
    {% if product_image %}  
      
    {% endif %}  
    <div class="product-meta">  
        <h2>{{ product_name }}</h2>  
        <div style="margin-top:8px; display:flex; align-items:center; flex-wrap:wrap; gap:6px;">  
            <div class="rating-badge">&#11088; {{ rating }} / 5</div>  
            {% if platform and platform != 'unknown' %}  
            <span class="source-badge {{ platform }}">  
                {% if platform == 'amazon' %} 📦 Analysed from Amazon  
                {% elif platform == 'flipkart' %} 📦 Analysed from Flipkart  
                {% elif platform == 'myntra' %} 📦 Analysed from Myntra  
                {% elif platform == 'snapdeal' %} 📦 Analysed from Snapdeal  
                {% endif %}  
            </span>  
            {% endif %}  
        </div>  
    </div>  
</div>
```

```
<div class="sentiment-grid">  
    <div class="sentiment-box positive">  
        Positive <span>{{ result.positive }}</span>  
    </div>  
    <div class="sentiment-box negative">  
        Negative <span>{{ result.negative }}</span>
```

```

</div>

<div class="sentiment-box neutral">
    Neutral <span>{{ result.neutral }}</span>
</div>

</div>

<div class="graphs">
    
    
</div>

{% set skw = result.sentiment_keywords %}
{% if skw %}
<div class="kw-section">
    <div class="kw-section-title">Keywords Driving Each Sentiment</div>
    <div class="kw-grid">
        <div class="kw-group positive">
            <div class="kw-group-label"><span class="kw-dot"></span>Positive</div>
            <div class="kw-tags">
                {% if skw.positive %}
                    {% for kw in skw.positive %}
                        <span class="kw-tag">{{ kw }}</span>
                    {% endfor %}
                {% else %}
                    <span class="kw-empty">No keywords found</span>
                {% endif %}
            </div>
        </div>
        <div class="kw-group negative">
            <div class="kw-group-label"><span class="kw-dot"></span>Negative</div>
            <div class="kw-tags">
                {% if skw.negative %}
                    {% for kw in skw.negative %}
                        <span class="kw-tag">{{ kw }}</span>
                    {% endfor %}
                {% else %}
                    <span class="kw-empty">No keywords found</span>
                {% endif %}
            </div>
        </div>
        <div class="kw-group neutral">

```

<div class="kw-group-label">Neutral</div>

<div class="kw-tags">

{% if skw.neutral %}

{% for kw in skw.neutral %}

{{ kw }}

{% endfor %}

{% else %}

No keywords found

{% endif %}

</div>

</div>

</div>

</div>

{% endif %}

{% if impact %}

<div class="sales-section">

<h3>Sales Impact Analysis</h3>

<div class="sales-metrics">

<div class="sales-card before">

Before Bad Reviews

{{ impact.sales_before }}

units

</div>

<div class="sales-card after">

After Bad Reviews

{{ impact.sales_after }}

units

</div>

<div class="sales-card drop">

Sales Drop

{{ impact.drop_pct }}%

decline

</div>

<div class="sales-card verdict {% if impact.severity == 'High' %}high{% elif impact.severity == 'Critical'

%}critical{% endif %}>

Impact Severity

{{ impact.severity }}

</div>

</div>

<div class="sales-chart">


```

</div>

</div>
{% endif %}

<div class="reviews-section">
  <h3>Customer Reviews {% if total_reviews %}({{ [total_reviews, 10] | min }} shown){% endif %}</h3>
  {% for r in reviews %}
    <div class="review-card">
      <div class="review-number">{{ loop.index }}</div>
      <div class="review-content">
        <div class="review-name">{{ r.name }}</div>
        <div class="review-text">{{ r.text }}</div>
      </div>
    </div>
  {% endfor %}
</div>

{% endif %}
</div>

</body>
</html>

```

CONCLUSION

This project successfully presents a comprehensive and automated system for extracting and analyzing customer reviews from multiple e-commerce platforms such as Amazon, Flipkart, Myntra, and Snapdeal. In the rapidly growing digital marketplace, customer reviews play a vital role in influencing purchasing decisions and determining product success. However, analyzing a large volume of unstructured review data manually is both time-consuming and inefficient. This project effectively addresses this challenge by providing a smart and automated solution.

The system integrates web scraping techniques to collect real-time product reviews and applies Natural Language Processing (NLP) methods to analyze customer sentiment. Using the VADER sentiment analysis model, reviews are classified into positive, negative, and neutral categories, enabling users to quickly understand the overall perception of a product. The keyword extraction feature further enhances the analysis by identifying commonly used words, which helps in highlighting frequent issues such as product defects, delivery delays, or quality concerns.

A significant contribution of this project is the inclusion of a sales impact prediction module. By analyzing the proportion of negative and neutral reviews, the system estimates the potential decline in product sales and categorizes the severity of the impact. This provides valuable insights for businesses, allowing them to take corrective actions and improve customer satisfaction. The ability to connect customer sentiment with business performance makes this system more practical and useful compared to traditional review analysis tools.

The project also emphasizes data visualization by presenting results through pie charts and bar graphs. These graphical representations make it easier for users to interpret the data and identify patterns without analyzing raw text. The use of modern technologies such as Flask for backend development, Selenium for web scraping, NLTK for sentiment analysis, and Matplotlib for visualization ensures that the system is efficient, scalable, and reliable.

Overall, the project demonstrates the effective application of data analysis and NLP techniques in solving real-world problems. It transforms raw customer feedback into actionable insights that benefit both consumers and businesses. Customers can make informed purchasing decisions, while businesses can monitor product performance and improve their services based on feedback.

In conclusion, this system provides a strong foundation for automated review analysis and business insight generation. Future improvements can include the integration of advanced machine learning and deep learning models for higher accuracy, multilingual support for analyzing reviews in different languages, and real-time dashboards for continuous monitoring. With further enhancements, this project has the potential to be developed into a full-scale commercial application.

.

REFERENCES

1. S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
2. C.J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2014.
3. M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA, USA: O'Reilly Media, 2013.
4. A. Sweigart, *Automate the Boring Stuff with Python*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2019.
5. Selenium Documentation, "Selenium WebDriver Documentation."
6. M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
7. J.D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
8. Wes McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
9. NLTK Documentation, "Natural Language Toolkit Documentation."
10. A. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
11. E-commerce Platforms Documentation: Amazon, Flipkart, Myntra, and Snapdeal product review pages (used for data extraction and testing).